

SSH Hardening and Intrusion Prevention

Server Engineering Labs

Author:

Laura Ortiz Moreno

Topics Covered:

- SSH server configuration
- Key-based authentication
- Port hardening
- Firewall management
- Fail2ban intrusion prevention
- Secure remote administration

Environment:

Debian Linux Virtual Machine
Rocky Linux Client VM
Virtualized lab environment

Contents

1	Introduction	2
2	SSH Service Verification	2
3	Changing the Default SSH Port	2
3.1	Firewall Configuration	3
4	SSH Key Authentication	3
4.1	Security Considerations	4
5	Disabling Password Authentication	4
6	Fail2ban Installation and Configuration	4
6.1	SSH Jail Configuration	5
6.2	Monitoring Banned Attempts	5
7	Remote Access Validation	6
8	Challenges Encountered	6
9	Conclusion	6

1. Introduction

This report documents the configuration and hardening of SSH services in a Linux server environment.

The goal of the laboratory was to:

- Configure secure remote access
- Replace password authentication with SSH keys
- Reduce exposure to automated attacks
- Configure firewall rules
- Deploy Fail2ban for intrusion prevention

The exercises were performed in a client-server virtualized environment using Debian and Rocky Linux virtual machines.

2. SSH Service Verification

The first step consisted of verifying that the SSH service was installed and active.

Useful commands included:

```
sudo systemctl status ssh
sudo ss -tulpn
```

These commands allow administrators to:

- Check whether the SSH daemon is running
- Verify listening ports
- Inspect active network services

3. Changing the Default SSH Port

To reduce exposure to automated scans and brute-force attacks, the default SSH port was modified.

The SSH configuration file was edited:

```
sudo nano /etc/ssh/sshd_config
```

The default port:

```
Port 22
```

was replaced with a custom port:

```
Port 22022
```

```

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 22022
AddressFamily any
ListenAddress 0.0.0.0
ListenAddress ::

HostKey /etc/ssh/ssh_host_rsa_key
HostKey /etc/ssh/ssh_host_ecdsa_key
HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
SyslogFacility AUTH
LogLevel INFO

# Authentication:

LoginGraceTime 2m
PermitRootLogin no
StrictModes yes
MaxAuthTries 6
MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

AuthorizedPrincipalsFile none

```

Figure 1: Customized SSH daemon configuration using a non-default port and hardened authentication settings.

The SSH service was then restarted:

```
sudo systemctl restart ssh
```

3.1. Firewall Configuration

Firewall rules were updated to allow connections through the new port.

Example using UFW:

```
sudo ufw allow 22022/tcp
sudo ufw reload
```

4. SSH Key Authentication

Key-based authentication was configured to improve security and avoid password-based login.

SSH keys were generated on the client machine:

```
ssh-keygen -t ed25519
```

The public key was copied to the server:

```
ssh-copy-id -p 22022 user@server
```

A successful login test was then performed:

```
ssh -p 22022 user@server
```

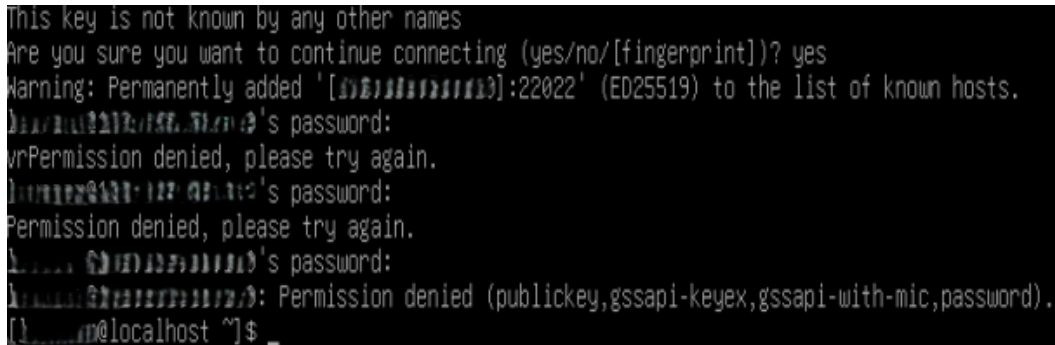
A terminal window showing the output of an SSH connection attempt. The text is as follows:
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[REDACTED]:22022' (ED25519) to the list of known hosts.
[REDACTED]'s password:
Permission denied, please try again.
[REDACTED]'s password:
Permission denied, please try again.
[REDACTED]'s password:
[REDACTED]: Permission denied (publickey,gssapi-keyex,gssapi-with-mic,password).
[REDACTED]@localhost ~]\$

Figure 2: SSH authentication test performed against the hardened SSH service using the custom port configuration.

4.1. Security Considerations

Benefits of SSH keys include:

- Stronger authentication
- Protection against password brute-force attacks
- Easier automation of administrative tasks

5. Disabling Password Authentication

After verifying that key authentication worked correctly, password authentication was disabled.

The following parameters were modified in:

```
/etc/ssh/sshd_config
```

```
PasswordAuthentication no  
PermitRootLogin no
```

The service was restarted afterward:

```
sudo systemctl restart ssh
```

6. Fail2ban Installation and Configuration

Fail2ban was installed to automatically block repeated failed login attempts.

Installation:

```
sudo apt install fail2ban
```

The service status was verified:

```
sudo systemctl status fail2ban
```

6.1. SSH Jail Configuration

A local configuration file was created:

```
sudo nano /etc/fail2ban/jail.local
```

Example configuration:

```
[sshd]
enabled = true
port = 22022
maxretry = 5
bantime = 10m
findtime = 10m
```

6.2. Monitoring Banned Attempts

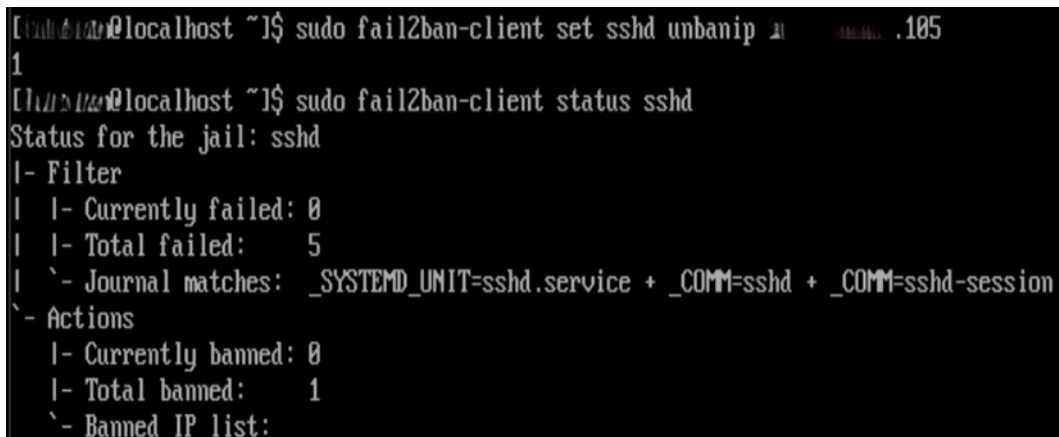
Fail2ban status commands:

```
sudo fail2ban-client status
sudo fail2ban-client status sshd
```



```
[11:11:30@localhost ~]$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 5
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd + _COMM=sshd-session
`-- Actions
    |- Currently banned: 1
    |- Total banned: 1
    `-- Banned IP list: 192.168.1.105
```

Figure 3: Fail2ban jail status showing detection and automatic banning of repeated failed SSH login attempts.



```
[11:11:30@localhost ~]$ sudo fail2ban-client set sshd unbanip 192.168.1.105
1
[11:11:30@localhost ~]$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 5
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd + _COMM=sshd-session
`-- Actions
    |- Currently banned: 0
    |- Total banned: 1
    `-- Banned IP list:
```

Figure 4: Manual unban operation and updated Fail2ban SSH jail status after removing the blocked client.

7. Remote Access Validation

Several validation tests were performed:

- SSH login through the custom port
- Key-only authentication
- Verification of blocked failed attempts
- Firewall validation

The hardened configuration successfully reduced exposure while maintaining remote administration functionality.

8. Challenges Encountered

Several configuration issues appeared during the lab:

- Incorrect firewall rules blocking SSH access
- SSH restart failures due to configuration syntax errors
- Key permission problems
- Fail2ban misconfiguration during testing

Special care was required because incorrect SSH settings may completely block remote access to the server.

9. Conclusion

This laboratory provided practical experience in Linux server hardening and secure remote administration.

The exercises demonstrated:

- SSH configuration and troubleshooting
- Secure authentication using SSH keys
- Firewall configuration
- Basic intrusion prevention with Fail2ban
- Security-oriented system administration practices

These concepts are fundamental in Linux infrastructure administration, cybersecurity, and production server environments.